UNIVERSIDADE FEDERAL DE SERGIPE

COMP0472 - Sistemas Operacionais - 2026.2

Virtualização, contêineres, namespaces e cgroups

Atividade 3 (ATV3)

Álex Felipe Santos Melo - 202200025395

Prof. Glauco de Figueiredo Carneiro - UFS

Contexto e Objetivo

Pergunta norteadora

Como namespaces alteram a visão de processos, mounts e rede, e como cgroups limitam recursos sem transformar um contêiner em uma máquina virtual independente?

O que foi feito

1. Registrar o ambiente real do Docker no notebook (WSL2, kernel, cgroup v2)
2. Comparar os namespaces de dois contêineres da mesma imagem
3. Medir o mesmo trabalho sob 0,5 e 1,0 CPU de quota, 3 vezes cada
4. Testar leitura e escrita numa montagem somente leitura

Ambiente e Camadas

Ambiente de execução

WSL2 - Ubuntu 26.04 LTS

Windows 11 com Hyper-V

AMD Ryzen 7 7735HS

4 CPUs e 7,8 GiB para o WSL2

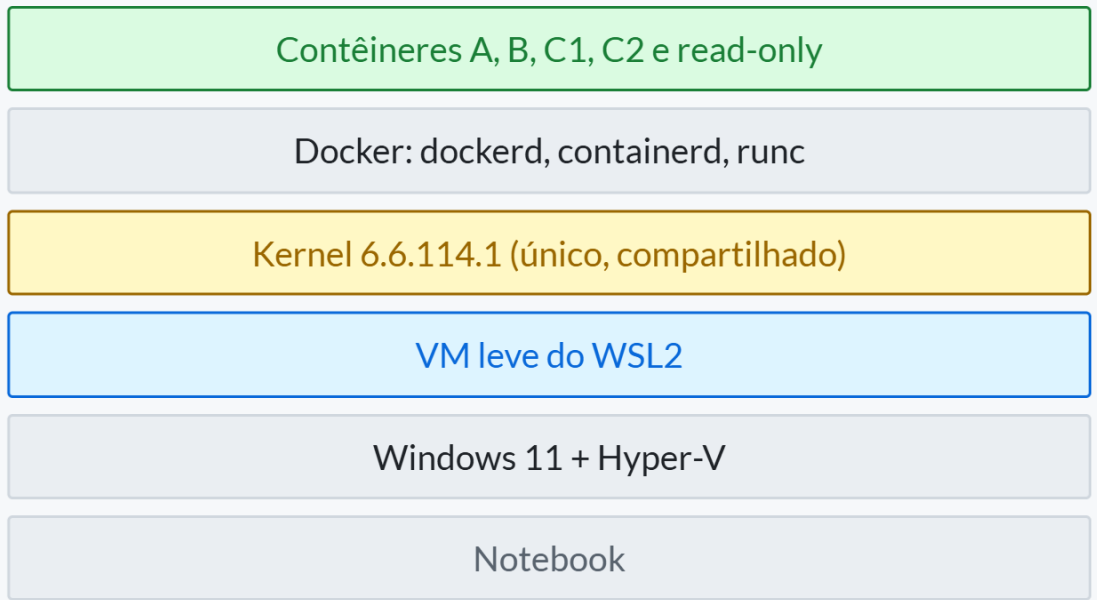
Docker Engine 29.1.3

cgroup v2, driver systemd

python:3.12-slim

fixada por sha256:5e2bdb1b0d54...

Camadas, de cima para baixo



Namespaces: Dois Contêineres, Visões Separadas

Mesma imagem, dois contêineres ativos, handles lidos com readlink /proc/1/ns/*

Recurso	Contêiner A	Contêiner B	Iguais?	O que muda
pid	pid:[4026532297]	pid:[4026532361]	não	cada um vê só o seu PID 1
mnt	mnt:[4026532294]	mnt:[4026532358]	não	raiz overlay própria
net	net:[4026532299]	net:[4026532363]	não	só a interface lo
cgroup	cgroup:[4026532298]	cgroup:[4026532362]	não	cgroup v2: namespace privado

Terminal do host

pid:[4026532219], mnt:[4026532217] e net:[4026531840], diferentes dos dois contêineres

Lido de fora

/proc/4422/ns/* no host dá os mesmos handles de A: é o mesmo processo visto de dois lugares

Os Dois São PID 1, e Não São o Mesmo Processo

Contêiner A, lido em /proc/4422/status

```
NSpid: 4422 1
```

4422 no host, 1 dentro do contêiner

Contêiner B, lido em /proc/4476/status

```
NSpid: 4476 1
```

outro processo, também 1 na sua tabela

Analogia: senha de atendimento

A senha 1 do caixa A e a senha 1 do caixa B são pessoas diferentes. O PID do host é o CPF: único no prédio todo.

O que os handles não provam

Kernel separado, limite de recurso ou usuário separado: sem user namespace, root no contêiner é root no host.

Configurações do Experimento de cgroups

run_experimentos.sh

```
# cada execucao (C1 = 0.5, C2 = 1.0)
docker run --cpus 0.5 --memory 256m --memory-swap 256m --
pids-limit 64 --network none --user 65534 --cap-drop ALL
python@sha256:5e2b... carga_cpu.py --n 40000000
# limite efetivo lido no kernel
cat /sys/fs/cgroup/cpu.max
cat /sys/fs/cgroup/cpu.stat
```

C1

Quota de 0,5 CPU

50 ms de CPU a cada 100 ms

C2

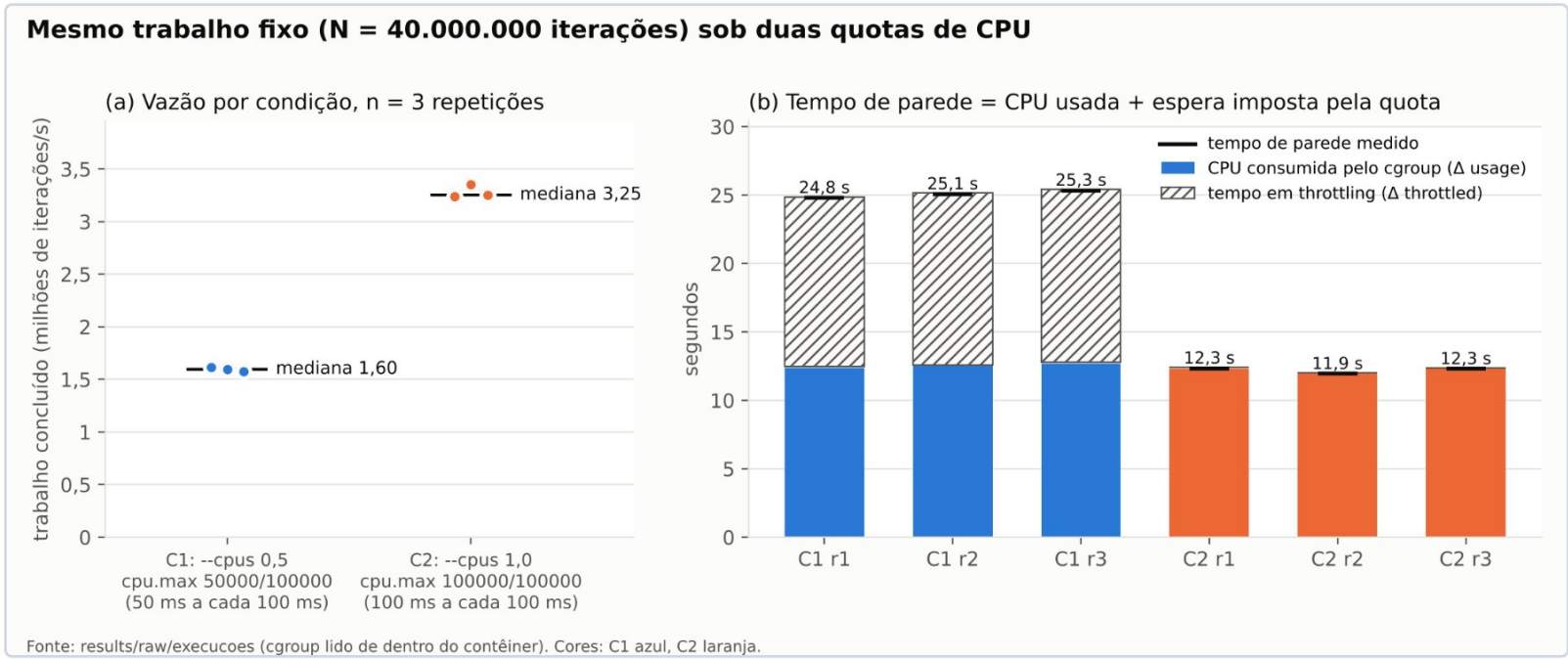
Quota de 1,0 CPU

mesmos limites, imagem e trabalho

Protocolo

Piloto com 2 milhões; N = 40 milhões congelado. 3 repetições por condição, ordem C1 C2, C2 C1, C1 C2. Checksum 098953c3ffd76aa3 conferido fora do contêiner.

Resultados: Metade da Quota, Metade da Vazão



25,07 s

C1 parede (mediana)

12,30 s

C2 parede (mediana)

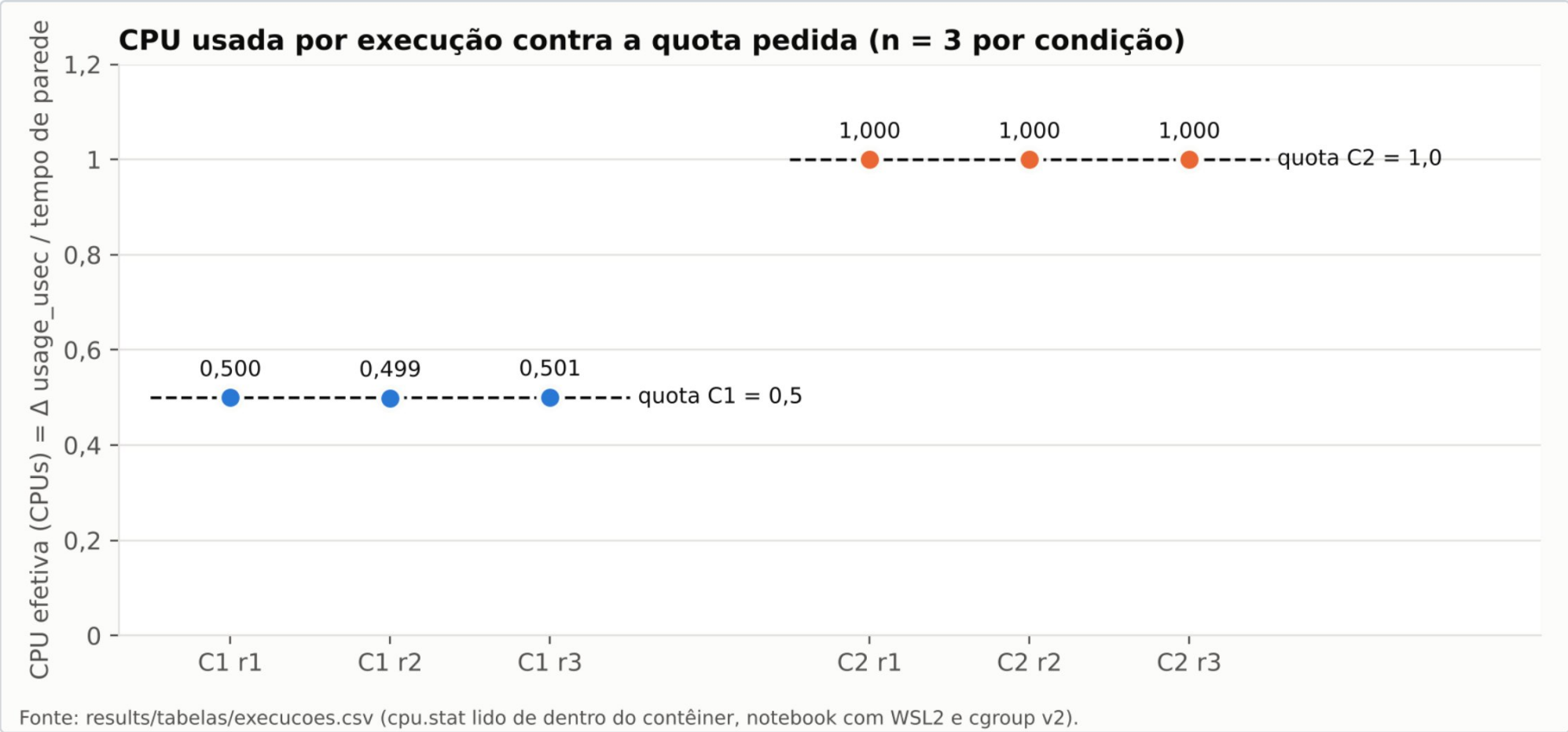
2,04

razão das vazões C2 / C1

100%

períodos de C1 com throttling

CPU Usada x Quota: o Kernel Cumpre à Risca



Cada ponto é uma execução: CPU consumida pelo cgroup dividida pelo tempo de parede. Todos caem em cima da linha da quota.

Como Ler a Quota de CPU

0,500

CPU usada em média por C1

CPU consumida 12,52 s + espera 12,54 s = parede 25,07 s

cpu.max = 50000 100000

50 ms de CPU a cada 100 ms, somados no grupo. Não é núcleo reservado nem afinidade.

Por que C2 quase não travou

A carga tem uma thread e nunca passa de 1 CPU. Só 6 períodos, 3,2 ms no total, numa repetição.

Por que deu 2,04 e não 2,00

C1 gastou 1,8% mais CPU para o mesmo trabalho. Na nuvem foram 7,4% e razão 2,14: ruído do host.

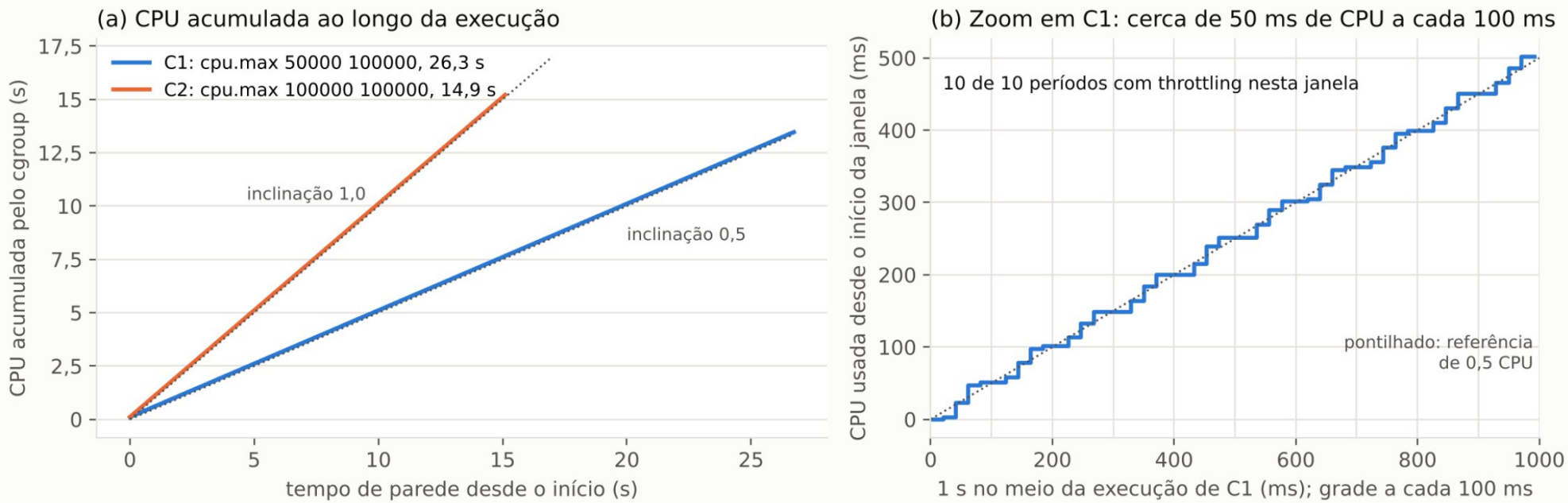
Ameaça principal à validade: ruído do notebook (Windows rodando junto), n = 3 e carga de uma thread só

Linha do Tempo do Throttling

Experimento extra

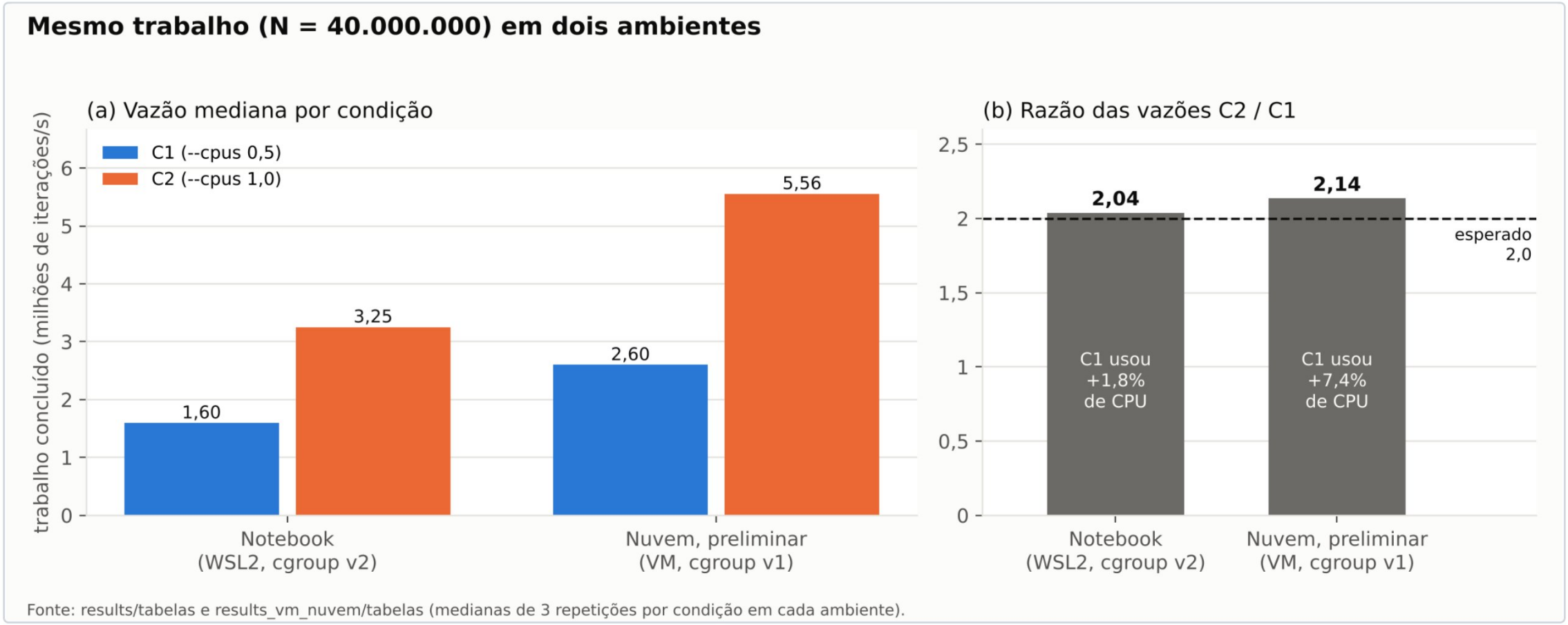
Por que fiz: as 6 execuções só dão o total antes e depois. Elas não dizem se a quota deixa a CPU mais lenta ou se liga e desliga o processo. Li o cpu.stat pelo host a cada 20 ms, uma vez em C1 e uma em C2.

Linha do tempo do throttling (cpu.stat lido pelo host a cada 20 ms)



Fonte: results/linha_do_tempo (uma execução por condição, mesmo N = 40.000.000, notebook com WSL2 e cgroup v2).

Notebook x Nuvem: o Mesmo Comportamento



A vazão absoluta muda com a máquina; a razão fica perto de 2 nos dois. O notebook é a medição oficial; a nuvem foi a execução preliminar.

Montagem Read-only e Menor Privilégio

dados_demo/texto.txt montado como /dados:ro, sem --privileged e sem rede

Caso	Quem tenta	Montagem	Leitura	Escrita	Quem negou
D1	uid 65534, sem capabilities, no_new_privs	/dados ro	ok	EROFS	a montagem
D2	root, capabilities padrão	/dados ro	ok	EROFS	a montagem
D3	uid 65534, sem capabilities	/controle rw	não testada	EACCES	a permissão

Vitrine trancada

Read-only é a vitrine: nem o diretor do museu (root) escreve. Permissão é a sala sem chave: barra por outro motivo.

Não é isolamento absoluto

O processo ainda lê tudo o que foi montado, e o kernel continua sendo o mesmo para todos.

Conclusão

01 Namespaces separaram PID, mount e rede: o mesmo processo é 4422 no host e 1 no contêiner

02 cgroups limitaram de verdade: 0,500 CPU, throttling em todos os períodos, 25,07 s contra 12,30 s

03 Tudo sobre um único kernel, o da VM do WSL2: contêiner não é VM

04 Em inferência: teto de RAM, limite de tarefas e corpus read-only ajudam, mas kernel compartilhado e VRAM seguem como risco